

Deep Research: Avatrada 57 Topic 005

Engineering Report: Composite Sentiment Scoring Engine

Report ID: ETR-2023-09-CSS **Date:** October 26, 2023 **Author:** Autonomous Technical Researcher **Subject:** Deep-Dive Analysis of a Weighted News and Social Sentiment Aggregation System

Executive Summary

This report provides a detailed engineering analysis of the "Sentiment Composite Scoring" component specified in the source documentation. The system's objective is to create a unified sentiment signal by aggregating scores from disparate sources: professional news (Benzinga Pro) and social media (Reddit/StockTwits).

The core technical challenges are the normalization of fundamentally different sentiment scales (a statistical Z-score vs. a lexicon-based VADER score) into a unified `[-1.0, +1.0]` range and the robust handling of intermittent or missing data streams.

This analysis deconstructs the system architecture, presents a concrete Python-based implementation strategy for normalization and aggregation, and conducts a critical review of potential failure modes, edge cases, and optimization opportunities. The proposed architecture is technically sound and provides a robust framework for generating a high-fidelity, composite sentiment signal.

1.0 Technical Deconstruction

The system is a data processing pipeline designed to ingest, standardize, and combine multiple sentiment indicators into a single, actionable score.

1.1 Core Architecture

The architecture can be broken down into three primary stages:

1. **Ingestion:** Raw data is fetched from source APIs.

- **News Sentiment:** Benzinga Pro API provides a structured, likely proprietary, sentiment score. The prompt references a Z-score, which measures the number of standard deviations a data point is from the mean of a reference dataset.
- **Social Sentiment:** Raw text data (posts, comments) is fetched from Reddit and StockTwits APIs. This unstructured data requires an internal sentiment analysis model to generate a score. VADER is specified as a candidate model.

2. **Normalization:** The heterogeneous scores from the ingestion layer are transformed onto a common, bounded mathematical space.

- **Benzinga Z-score:** Unbounded, theoretically ranging from $(-\infty, +\infty)$. In practice, values rarely exceed $[-4, +4]$.
- **VADER Compound Score:** Loosely bounded, typically $[-1, +1]$, but can slightly exceed these bounds due to linguistic intensifiers.
- **Target:** A strict, normalized range of $[-1.0, +1.0]$ for both streams.

3. **Aggregation:** The normalized scores are combined using a predefined weighted average to produce the final Composite Score.

- **Formula:** $\text{Score} = (0.6 * \text{News_Normalized}) + (0.4 * \text{Social_Normalized})$

1.2 The Normalization Challenge

Directly applying the aggregation formula to raw scores is mathematically invalid. For example, a highly significant Benzinga Z-score of `+3.0` and a very positive VADER score of `+0.95` are on completely different scales. A robust normalization function is required to make them comparable.

- **Z-score Problem:** A linear mapping (like Min-Max scaling) is not suitable for an unbounded source like a Z-score, as a new extreme value would break the scaling. A non-linear "squashing" function is required.
- **VADER Problem:** While close to the target range, its bounds are not guaranteed. A simple clipping mechanism is needed to enforce the `[-1, +1]` constraint.

1.3 Data Handling Logic

The system must be resilient to API failures or a lack of relevant content for a given asset. If the Benzinga API is down or there are no new Reddit posts about a specific stock, the composite score calculation must adapt rather than fail. The specification requires explicit logic to manage these scenarios.

2.0 Implementation Strategy

This section outlines a practical implementation using Python, addressing the core challenges of normalization and data handling.

2.1 System Prerequisites & Dependencies

- **APIs:** Access and API keys for Benzinga Pro, Reddit (PRAW), and StockTwits.
- **Python Libraries:**
 - `requests` or `httpx` : For making API calls.
 - `numpy` : For numerical operations, specifically `np.tanh` .
 - `nltk` : For VADER sentiment analysis. (Requires one-time download of the VADER lexicon: `nltk.download('vader_lexicon')`).

- `pandas` : Useful for handling and aggregating time-series data, though not strictly required for the core function.

2.2 Normalization Functions

To solve the normalization challenge, we will implement two distinct functions.

2.2.1 Normalizing the Benzinga Z-score

A hyperbolic tangent function (`tanh`) is an ideal choice for squashing an unbounded value into a `(-1, 1)` range. It provides a non-linear mapping where the output's sensitivity decreases as the input moves further from zero, effectively handling outliers gracefully.

We introduce a scaling factor `k` to control the function's sensitivity. A reasonable starting point is to assume that a Z-score of `~3.0` represents a very strong signal, which we want to map close to `+1.0`.

Mathematical Formula:

$$\text{Normalized_Score} = \tanh(\text{Z_Score} / k)$$

Where `k` is a calibration constant (e.g., `k=3`).

Python Implementation:

```
import numpy as np

def normalize_z_score(z_score: float, k: float = 3.0) -> float:
    """
    Normalizes a Z-score to a [-1.0, 1.0] range using a scaled tanh function.

    Args:
        z_score: The raw Z-score from the Benzinga API.
        k: A scaling factor to adjust sensitivity. A lower k makes the
           function saturate faster. k=3 implies a Z-score of 3 is considered
           a very strong signal.
```

```
Returns:
    A normalized score between -1.0 and 1.0.
    """
    if z_score is None:
        return None
    return np.tanh(z_score / k)
```

2.2.2 Normalizing the VADER Score

Since the VADER compound score is already close to the desired `[-1, 1]` range, a simple clipping function is sufficient and computationally cheap.

Python Implementation:

```
def normalize_vader_score(vader_score: float) -> float:
    """
    Normalizes a VADER score by clipping it to the strict [-1.0, 1.0] range.

    Args:
        vader_score: The raw compound score from a VADER analysis.

    Returns:
        A clipped score guaranteed to be between -1.0 and 1.0.
    """
    if vader_score is None:
        return None
    return max(-1.0, min(1.0, vader_score))
```

2.3 Composite Scoring Function with Data Handling

The main function integrates the normalization logic and the strategy for handling missing data. The most robust strategy for missing streams is to re-distribute the weights to the available streams.

Python Implementation:

```
def calculate_composite_sentiment(news_z_score: float | None,
    social_vader_score: float | None) -> float | None:
```

```
"""
```

```
Calculates a composite sentiment score from news and social data streams.
```

```
Specification:
```

- Weighted aggregation: News (60%), Social (40%).
- Normalizes scores to a [-1.0, 1.0] range.
- Handles missing data streams by re-weighting the available data.

```
Args:
```

```
    news_z_score: The raw Z-score from the Benzinga API. Can be None.
```

```
    social_vader_score: The raw VADER score from social media. Can be None.
```

```
Returns:
```

```
    The composite sentiment score, or None if no data is available.
```

```
"""
```

```
# Step 1: Normalize the raw scores
```

```
norm_news = normalize_z_score(news_z_score)
```

```
norm_social = normalize_vader_score(social_vader_score)
```

```
# Step 2: Handle missing data streams and apply weights
```

```
if norm_news is not None and norm_social is not None:
```

```
    # Both streams are available: apply standard weights
```

```
    composite_score = (0.6 * norm_news) + (0.4 * norm_social)
```

```
elif norm_news is not None:
```

```
    # Only news is available: use 100% weight for news
```

```
    composite_score = norm_news
```

```
elif norm_social is not None:
```

```
    # Only social is available: use 100% weight for social
```

```
    composite_score = norm_social
```

```
else:
```

```
    # No data available
```

```
    return None
```

```
# Final clipping to ensure the result is strictly within bounds
```

```
return max(-1.0, min(1.0, composite_score))
```

```
# --- Example Usage ---
```

```
# Scenario 1: Both data streams are present
```

```
news_score_1 = 2.5 # Strong positive news
```

```
social_score_1 = 0.8 # Strong positive social chatter
```

```
composite_1 = calculate_composite_sentiment(news_score_1, social_score_1)
# norm_news = tanh(2.5/3) = 0.68; norm_social = 0.8
# composite = (0.6 * 0.68) + (0.4 * 0.8) = 0.408 + 0.32 = 0.728
print(f"Scenario 1 (All data): {composite_1:.4f}")

# Scenario 2: News stream is missing (API down)
news_score_2 = None
social_score_2 = -0.5 # Moderately negative social chatter
composite_2 = calculate_composite_sentiment(news_score_2, social_score_2)
# composite = norm_social = -0.5
print(f"Scenario 2 (News missing): {composite_2:.4f}")

# Scenario 3: No data available
composite_3 = calculate_composite_sentiment(None, None)
print(f"Scenario 3 (No data): {composite_3}")
```

3.0 Critical Analysis

A robust system requires an understanding of its limitations and potential points of failure.

3.1 Potential Failure Modes & Edge Cases

- **API Rate Limiting & Downtime:** The re-weighting logic handles complete downtime, but intermittent failures or rate limiting could introduce data staleness. The system should have a timestamping and TTL (Time-To-Live) mechanism for scores to avoid using old data.
- **Normalization Saturation:** The `tanh` function saturates near `+1` and `-1`. If Benzinga Z-scores regularly exceed the `k` value (e.g., Z-scores of `+5` or `+6` become common), the model will lose its ability to differentiate between "very strong" and "extremely strong" signals. The distribution of Z-scores should be monitored over time to recalibrate `k` if necessary.
- **Garbage In, Garbage Out (GIGO):**
 - **Social Sentiment:** VADER is fast but struggles with sarcasm, complex financial jargon, and context. A post like "Wow, \$XYZ to the

moon! ☐ Can't wait for it to hit \$0" is sarcastic and would likely be misinterpreted as positive.

- **News Sentiment:** News APIs can mis-tag articles or assign sentiment to a company mention that is only incidental to the main story.
- **Concept Drift:** The meaning of words and the nature of news reporting can change over time. A model (both Benzinga's and VADER) trained on historical data may see its performance degrade. The system requires periodic evaluation and potential retraining or fine-tuning.

3.2 Performance & Scalability Considerations

- **API Latency:** The primary bottleneck for real-time performance will be the latency of the external APIs. For scaling to hundreds or thousands of assets, API calls should be made asynchronously (e.g., using `asyncio` and `aiohttp`).
- **Computational Cost:** VADER is very lightweight. The normalization and aggregation logic is trivial. The main computational load is on the I/O for API requests.
- **Data Storage:** Storing the resulting time-series of composite scores would require a suitable database, such as a time-series database (e.g., InfluxDB, TimescaleDB) for efficient querying and analysis.

3.3 Potential Optimizations & Future Enhancements

- **Dynamic Weighting:** The fixed `60/40` split is a strong starting point but may not be optimal under all market conditions. An advanced implementation could dynamically adjust weights based on:
 - **Volume:** Increase the weight of the social score when social media volume (number of posts/comments) for an asset spikes.
 - **Volatility:** In times of high market volatility, news sentiment might be a more reliable (less noisy) signal, justifying a higher weight.
- **Advanced NLP Models:** For social sentiment, replacing VADER with a more sophisticated, context-aware transformer model (e.g., a fine-tuned

FinBERT) could significantly improve accuracy, at the cost of higher computational requirements.

- **Alternative Normalization:** Instead of a static `tanh` scaling, one could use a rolling statistical normalization (e.g., calculating the Z-score of the VADER scores over a 30-day window). This makes the system self-adapting but can be less stable and harder to interpret.
- **Source Credibility Weighting:** Within the social score, not all sources are equal. A comment from a Reddit user with a long history of accurate predictions could be weighted more heavily than a new, anonymous account.